

Nov 18, 2025



Security Assessment **flipflop_proof_of_mint**

Professional Service

Table of Contents

1. Overview

- 1.1. Executive Summary
- 1.2. Project Summary
- 1.3. Assessment Summary
- 1.4. Assessment Scope

2. Checklist

3. Findings

- H-01: Lack of Authority Check for close token
- H-02: Strict Equality Check Breaks Refund Functionality
- M-03: Limited Use Cases for the Pause Function
- M-04: Overly Strict Check in process refund
- M-05: Incorrect init_pool_wsol_amount Calculation
- M-06: Pool Creation Can Be Front-Run
- L-07: Incorrect Configuration of init_pool_wsol_amount
- L-08: initialize_system and initialize_launch_rule Susceptible to Front-Running
- L-09: Lack of Upper Bound for Fee Rate
- L-10: Insufficient Validation
- I-11: Typographical Error
- I-12: Mismatch Between Code Logic and Error Message
- I-13: Unnecessary Check in mint_tokens Function
- I-14: Adherence to the Checks-Effects-Interactions Pattern
- I-15: Redundant Check
- I-16: Lack of Zero-Address Check

4. Disclaimer

1. Overview

1.1. Executive Summary

flipflop_proof_of_mint is a decentralized fair launch token program built on Solana with automated liquidity provision and Raydium integration. This report has been prepared for flipflop_proof_of_mint to discover issues and vulnerabilities in the source code of this project.

This project utilizes concepts related to "Proof of Mint," a new proposal for token distribution on the blockchain, which is detailed in the documentation here: https://github.com/flipflop-fun/proof_of_mint_docs/blob/main/doc/md/proof_of_mint_v1.1.md. Our audit scope included verifying that the smart contract code correctly implements the formulas and logic as specified in the provided documentation. Verification of the mathematical and logical correctness of the formulas and their derivations, as presented in the documentation, was considered out of scope for this audit.

Conducted by Static Analysis and Manual Review, we have identified 2 High, 4 Medium, 4 Low and 6 Informational issues in commit 5ea8290.

In commit 0ef6247, the project team resolved the vulnerabilities H-01, H-02, M-05, L-08, and L-09, along with all informational-level findings. The team acknowledged issues M-03, M-04, M-06, L-07, and L-10 but confirmed that the current implementation aligns with their business design; therefore, no changes will be made for these issues. For specific resolution details, please refer to the Alleviation section.

1.2. Project Summary

Project Name	flipflop_proof_of_mint
Platform	Solana
Language	Rust
Codebase	Audit 1: https://github.com/flipflop-fun/solana-program/tree/5ea82905722f9e6ba9c450060853ed68f623f927 Final Audit: https://github.com/flipflop-fun/solana-program/tree/0ef6247d8b6aff2ef6e7c5a1eea052f88c0f45a9

1.3. Assessment Summary

Delivery Date	Nov 18, 2025
Audit Methodology	Static Analysis, Manual Review

1.4. Assessment Scope

ID	File	File Hash
1	/src/instructions/delegate_value_manager.rs	ef8ab46945e048c07887cafbbbe92d3e
2	/src/instructions/initialize_system.rs	7480026dc8bb040a187c4012841b78a2
3	/src/constants.rs	ef16d70d501fff81aef2054378785747
4	/src/instructions/initialize_launch_rule.rs	30c5766260e8c48ae74575fea4ca5691
5	/src/instructions/mint_tokens.rs	aac9d3eb84575d99e081b33d68eda3f6
6	/src/lib.rs	165c51d4e22e5057585bb526cbeb5f48
7	/src/instructions/close_token.rs	cbd056522039641c44d47b8ce3adaff3
8	/src/instructions/burn_mint_token_vault.rs	6b10bddac32714219f1c858054f093c8
9	/src/instructions/proxy_swap_base_in_pool.rs	755b0015cd858b49ac1d0e2d40d63724
10	/src/instructions/proxy_swap_base_out_pool.rs	db0fcff102835fc8519500d9f3193e16
11	/src/instructions/refund.rs	289d95bc2b1dba1ba7423e9b41ea8b0c
12	/src/utils.rs	3068d00b09e290e6edacc169bba20d76
13	/src/errors.rs	40b5ffea15b8d240ad905ac9c774d14b
14	/src/states.rs	f42e3b621e133d2de37a8ccdda56c742
15	/src/instructions/update_system.rs	979f56099d003c4fc689e9a725bffd12
16	/src/instructions/proxy_create_pool.rs	8a8568b8cfd626315d4645394f019978
17	/src/instructions.rs	33212b1be5e1579fdf95687422cb4543
18	/src/instructions/proxy_withdraw_pool.rs	3b69870113a7095512e9f9a0cb42e7bf
19	/src/instructions/set_referrer_code.rs	edff84c8c37af7f105868c51a397168e
20	/src/instructions/initialize_token.rs	cb73ab31751e5a207f2507202a0f2983

ID	File	File Hash
21	/src/instructions/proxy_burn_lp_tokens.rs	7a56348a0eff41ef777e94ddf23422c2
22	/src/events.rs	61ba0e9a4ad608b818b669ba3cc64503
23	/src/instructions/revoke_update_metadata_authority.rs	eb74c5ee9fc06479026fb1cef5ef43d1
24	/src/instructions/update_token_metadata.rs	4d84eba0a5c238818cc1acaa2bd1cfde
25	/src/instructions/proxy_deposit_pool.rs	6580bc39db97b6f8f840458390f4a059

2. Checklist

2.1. Code Security

String UTF8 Handling	Unsafe Block	Integer Overflow
Input Validation	Improper Use of Unwrap	Frozen Money
Index out of bounds	Unchecked Owner Transfer	While Continue
Right-To-Left-Override Character	Unauthenticated Storage Access	Risk For Weak Randomness
Stack overflows	Missing Checks for Return Values	Arithmetic errors
Panics	Type Casting	Array Length Manipulation
Uninitialized Variable	Shadow Variable	Divide Before Multiply
Affected by Compiler Bug		

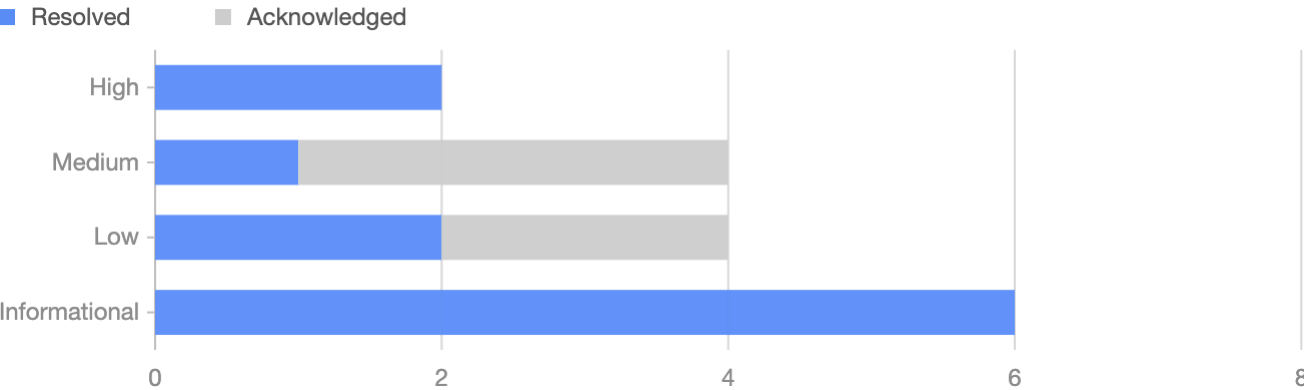
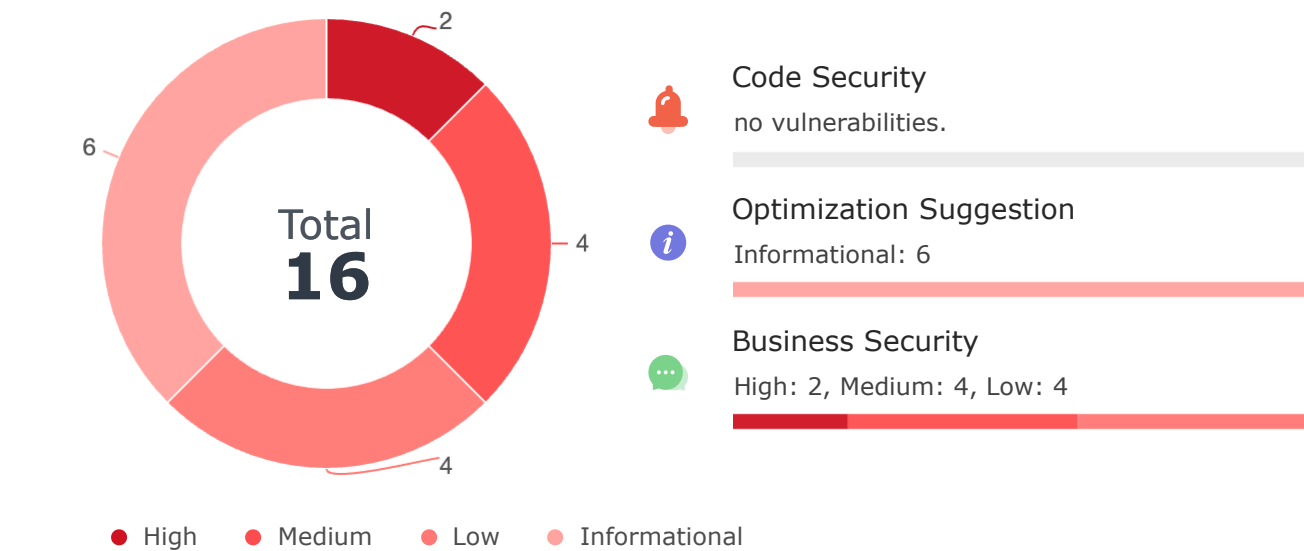
2.2. Optimization Suggestion

Compiler Version	Improper State Variable Modification
Function Visibility	Deprecated Function
Externally Controlled Variables	Code Style
Constant Specific	Event Specific
Return Value Unspecified	Inexistent Error Message
State Variable Specific	Import Issue
Compare With Timestamp/Block Number	Constructor in Base Contract Not Implemented
Compiler Error/Warning	Optimizable Return Statement
Unused Function	Code Complexity/Code Inefficiency
Loop Depends on Array Length	
Tautology Issue	
Redundant/Duplicated/Dead Code	
Undeclared Resource	
Unused Resource	

2.3. Business Security

The Code Implementation is Consistent With Comments, Project White Papers and Other Materials
Permission Check
Account Check

3. Findings



ID	Title	Category	Severity	Status
H-01	Lack of Authority Check for close_token	Business Security	High	Resolved
H-02	Strict Equality Check Breaks Refund Functionality	Business Security	High	Resolved
M-03	Limited Use Cases for the Pause Function	Business Security	Medium	Acknowledged
M-04	Overly Strict Check in process_refund	Business Security	Medium	Acknowledged
M-05	Incorrect init_pool_wsol_amount Calculation	Business Security	Medium	Resolved
M-06	Pool Creation Can Be Front-Run	Business Security	Medium	Acknowledged
L-07	Incorrect Configuration of init_pool_wsol_amount	Business Security	Low	Acknowledged
L-08	initialize_system and initialize_launch_rule Susceptible to Front-Running	Business Security	Low	Resolved

ID	Title	Category	Severity	Status
L-09	Lack of Upper Bound for Fee Rate	Business Security	● Low	Resolved
L-10	Insufficient Validation	Business Security	● Low	Acknowledged
I-11	Typographical Error	Optimization Suggestion	● Informational	Resolved
I-12	Mismatch Between Code Logic and Error Message	Optimization Suggestion	● Informational	Resolved
I-13	Unnecessary Check in mint_tokens Function	Optimization Suggestion	● Informational	Resolved
I-14	Adherence to the Checks-Effects-Interactions Pattern	Optimization Suggestion	● Informational	Resolved
I-15	Redundant Check	Optimization Suggestion	● Informational	Resolved
I-16	Lack of Zero-Address Check	Optimization Suggestion	● Informational	Resolved

H-01: Lack of Authority Check for close_token



High: Business Security

File Location: /src/instructions/close_token.rs:23

Description

In the `close_token` function, the identity of the `payer` is not strictly validated, which allows any user to call the `close_token` function to close the associated token accounts.

/src/instructions/close_token.rs

```
18  #[account(  
19      mut,  
20      constraint = payer.key() != Pubkey::default() @  
        ErrorCode::InvalidPayerAccount,  
21      constraint = payer.is_signer @ ErrorCode::PayerMustSign  
22  )]  
23  pub payer: Signer<'info>,
```

Recommendation

It is recommended to restrict the payer to the `config_account.admin`.

Alleviation

Resolved in commit 0ef6247.

H-02: Strict Equality Check Breaks Refund Functionality



High: Business Security

File Location: `/src/instructions/refund.rs:122`

Description

The `process_refund` function validates that the user's token account (`token_ata`) holds an amount strictly equal to `burn_amount_from_user` using an `==` operator. This implementation is fragile because a user's token account can receive transfers from any other account on the network. An external party—whether maliciously or accidentally—could transfer a small number of tokens to the user's `token_ata`.

This external deposit would cause the balance to exceed `burn_amount_from_user`, making the strict equality check fail. As a result, the `process_refund` transaction would permanently revert, preventing the legitimate user from being able to refund.

`/src/instructions/refund.rs`

```
107 fn process_refund(ctx: &mut Context<Refund>, _token_name: String,
108   _token_symbol: String) -> Result<()> {
109     let refund_data = &mut ctx.accounts.refund_account;
110     require!(refund_data.total_tokens > 0 && refund_data.vault_tokens >
111       0, ErrorCode::RefundTokensIsZero);
112
113     let token_ata = &mut ctx.accounts.token_ata;
114     let config_data = &mut ctx.accounts.config_account;
115     let mint = &mut ctx.accounts.mint;
116     let payer = &ctx.accounts.payer;
117     let token_program = &ctx.accounts.token_program;
118     let protocol_fee_account = ctx.accounts.protocol_fee_account;
119     to_account_info();
120     let system_config_data = &ctx.accounts.system_config_account;
121
122     let burn_amount_from_user = refund_data.total_tokens;
123     let burn_amount_from_vault = refund_data.vault_tokens;
124     require!(mint.supply >= burn_amount_from_user +
125       burn_amount_from_vault, ErrorCode::NotEnoughTokensToRefund);
126     require!(token_ata.amount == burn_amount_from_user,
127       ErrorCode::UserBalanceNotEqualToRefund);
128     require!(ctx.accounts.token_vault.amount >= burn_amount_from_vault,
129       ErrorCode::VaultBalanceNotEqualToRefund);
```

Recommendation

It is recommended to replace the strict equality check with a "greater than or equal to" (`>=`) comparison.

Alleviation

Resolved in commit 0ef6247.

M-03: Limited Use Cases for the Pause Function



Medium: Business Security

File Location:

Description

In this project, only the `initialize_token` function requires the project to be in a non-paused state. Other critical functions, such as `mint_tokens` and `refund`, do not perform any checks on the project's paused status. This lack of a comprehensive pause mechanism could prevent the team from stopping losses in a timely manner in the event the contract is compromised.

Recommendation

It is recommended to add a check for the paused state in critical functions, such as `mint_tokens` and `refund`.

Alleviation

The project team acknowledged the issue and decided to keep no change. They stated that, as a matter of principle, operations such as mint and refund should be irreversible and not subject to being halted by the platform once a token has been issued. This stance is taken to uphold the decentralization of the platform. Consequently, they have only implemented a pause control mechanism for the `initialize_token` function, deliberately omitting it from all other functions.

M-04: Overly Strict Check in process_refund



Medium: Business Security

File Location: /src/instructions/refund.rs:109

Description

In the `process_refund` function, there is a check that requires `refund_data.total_tokens > 0` && `refund_data.vault_tokens > 0`. However, according to the business logic, the refund operation should be executable as long as either of these two values is greater than zero. This overly strict check could cause the refund operation to fail.

/src/instructions/refund.rs

```
107 fn process_refund(ctx: &mut Context<Refund>, _token_name: String,
108   _token_symbol: String) -> Result<> {
109     let refund_data = &mut ctx.accounts.refund_account;
110     require!(refund_data.total_tokens > 0 && refund_data.vault_tokens >
111       0, ErrorCode::RefundTokensIsZero);
112
113     let token_ata = &mut ctx.accounts.token_ata;
114     let config_data = &mut ctx.accounts.config_account;
115     let mint = &mut ctx.accounts.mint;
116     let payer = &ctx.accounts.payer;
117     let token_program = &ctx.accounts.token_program;
118     let protocol_fee_account = ctx.accounts.protocol_fee_account.
119       to_account_info();
120     let system_config_data = &ctx.accounts.system_config_account;
```

Recommendation

It is recommended to change the condition to `refund_data.total_tokens > 0 || refund_data.vault_tokens > 0`.

Alleviation

The project team acknowledged the issue and decided to keep no change.

M-05: Incorrect init_pool_wsol_amount Calculation

Medium: Business Security



File Location:

/src/instructions/mint_tokens.rs:329,356,362

/src/utils.rs:331

Description

In the `process_mint_tokens` function, the `init_pool_wsol_amount` is first calculated based on the balance of the `wsol_vault`. Subsequently, the `graduate_fee` is transferred from the `wsol_vault` to the `protocol_wsol_vault`. Following this, the `create_pool` function is called to create the pool.

Within the `create_pool` function, the `init_pool_wsol_amount` is calculated again, also based on the balance of the `wsol_vault`. However, by this point, the balance of the `wsol_vault` has already decreased due to the fee transfer. This results in a new, smaller `init_pool_wsol_amount` that is inconsistent with the value originally calculated in `process_mint_tokens`.

This contradicts the intended business logic. The correct logic should be that the initial `init_pool_wsol_amount` calculated in `process_mint_tokens` is used as the definitive amount of WSOL liquidity to be added to the pool.

/src/instructions/mint_tokens.rs

```
328     let graduate_fee = if graduate_fee < ctx.accounts.  
        system_config_account.min_graduate_fee { ctx.accounts.  
        system_config_account.min_graduate_fee } else { graduate_fee };  
329     let init_pool_wsol_amount = (wsol_vault_balance as u128 * ctx.  
        accounts.system_config_account.init_pool_wsol_amount as u128 /  
        100_000) as u64;  
330     // msg!("graduate_fee: {}", graduate_fee);  
331     // msg!("wsol_vault_balance: {}", wsol_vault_balance);  
332     // msg!("raydium_cpmm_create_fee: {}", ctx.accounts.  
        system_config_account.raydium_cpmm_create_fee);  
333     // msg!("init_pool_wsol_amount: {}", init_pool_wsol_amount);  
334     require!(graduate_fee + ctx.accounts.system_config_account.  
        raydium_cpmm_create_fee + init_pool_wsol_amount <=  
        wsol_vault_balance, ErrorCode::GraduateFeeIsNotEnough);  
335  
336     let protocol_fee_account = ctx.accounts.protocol_fee_account.clone();  
337     let _wsol_ata_of_protocol_fee_account =  
        get_associated_token_address_with_program_id(  
338         &protocol_fee_account.key(),  
339         &WSOL,  
340         &ctx.accounts.token_program.key(),  
341     );  
342     let protocol_wsol_vault = ctx.accounts.protocol_wsol_vault.clone();  
343     require!(!protocol_wsol_vault.to_account_info().data_is_empty(),  
        ErrorCode::ProtocolWsolVaultIsIsEmpty);  
344     require!(  
        _wsol_ata_of_protocol_fee_account == protocol_wsol_vault.key()  
        ), ErrorCode::InvalidProtocolFeeAccount);
```

```

345
346     let transfer_instruction = Transfer {
347         from: ctx.accounts.wsol_vault.to_account_info(),
348         to: protocol_wsol_vault.to_account_info(),
349         authority: config_data.to_account_info(),
350     };
351     let context = CpiContext::new_with_signer(
352         ctx.accounts.token_program.to_account_info(),
353         transfer_instruction,
354         &config_signer,
355     );
356     transfer(context, graduate_fee)?;
357     msg!("Graduate fee transferred");
358 }
359
360 // 11- Check if liquidity pool created
361 if config_data.mint_state_data.current_era > config_data.target_eras &&
    ctx.accounts.pool_state.to_account_info().data_is_empty() {
362     create_pool(
363         config_data,
364         ctx.accounts.system_config_account.clone(),

```

/src/utils.rs

```

321     let wsol_vault_balance =
322         get_associated_token_account_balance(&wsol_vault.to_account_info())?;
323     let token_vault_balance =
324         get_associated_token_account_balance(&token_vault.to_account_info()
        ())?;
325     require!(wsol_vault_balance > 0, ErrorCode::InvalidVaultBalance);
326     // Caculate init pool wsol and tokens according to the mint price
327     // msg!("config_data.fee_rate: {}", config_data.fee_rate);
328     msg!("previous_mint_size_epoch: {}", previous_mint_size_epoch.unwrap() /
        1_000_000_000);
329     let current_mint_price = config_data.fee_rate /
        (previous_mint_size_epoch.unwrap() / 1_000_000_000); // cause the
        previous_mint_size_epoch is not including vault, the price is minter's
        cost
330     let initial_pool_price = current_mint_price * 105 / 100; // Increase ment
        rate of current mint price
331     let init_pool_wsol_amount = (wsol_vault_balance as u128 * system_config.
        init_pool_wsol_amount as u128 / 100_000) as u64;

```

Recommendation

It is recommended to pass the `init_pool_wsol_amount` calculated in function `process_mint_tokens` as an argument to `create_pool` instead of recalculating it.

Alleviation

Resolved in commit 0ef6247.

M-06: Pool Creation Can Be Front-Run



Medium: Business Security

File Location: /src/instructions/mint_tokens.rs:361

Description

When the "graduation" condition is met, the protocol is designed to create a new liquidity pool and add liquidity to it. However, an attacker can front-run this process by pre-emptively creating the pool before the graduation occurs. The attacker can then set the token price within this pool to a manipulated or incorrect level. Consequently, when the legitimate graduation transaction is executed, it will fail to create the pool because it already exists. Furthermore, the price in the attacker-created pool does not necessarily reflect the intended minting price, which disrupts the trading market.

/src/instructions/mint_tokens.rs

```
360 // 11- Check if liquidity pool created
361 if config_data.mint_state_data.current_era > config_data.target_eras &&
    ctx.accounts.pool_state.to_account_info().data_is_empty() {
362     create_pool(
363         config_data,
364         ctx.accounts.system_config_account.clone(),
365         ctx.accounts.mint.key(),
366         ctx.accounts.user.key(),
367         ctx.bumps.config_account,
368         ctx.accounts.wsol_vault.to_account_info(),
369         ctx.accounts.token_vault.to_account_info(),
370         ctx.accounts.user.to_account_info(),
371         ctx.accounts.destination.to_account_info(),
372         ctx.accounts.token_program.to_account_info(),
373         ctx.accounts.destination_wsol_ata.to_account_info(),
374         ctx.accounts.wsol_mint.to_account_info(),
375         ctx.accounts.system_program.to_account_info(),
376         &ctx.remaining_accounts.to_vec(),
377         Some(previous_mint_size_epoch),
378     )?;
379 }
```

Recommendation

It is recommended that the project team adopt the Token-2022 standard, pre-create the liquidity pool, and restrict trading on it until after the "graduation" event.

Alleviation

The project team acknowledged the issue and decided to keep no change. They stated that this issue is unavoidable with their current business design. Therefore, they permit any user to create the pool before the "graduation" event.

L-07: Incorrect Configuration of init_pool_wsol_amount

Low: Business Security



File Location:

/src/instructions/initialize_system.rs:38

/src/instructions/mint_tokens.rs:329,334

Description

In the `initialize_system` function, `system_config_data.init_pool_wsol_amount` is initialized to `100,000`. Then, during the graduation phase, the calculated `init_pool_wsol_amount` becomes equal to `wsol_vault_balance`. In cases where `graduate_fee` and `raydium_cpmm_create_fee` are not zero, the check `require!(graduate_fee + ctx.accounts.system_config_account.raydium_cpmm_create_fee + init_pool_wsol_amount <= wsol_vault_balance, ErrorCode::GraduateFeeIsNotEnough);` will fail.

/src/instructions/initialize_system.rs

```
27 pub fn initialize_system(ctx: Context<InitializeSystem>) -> Result<()> {
28     let system_config_data: &mut Account<'_, SystemConfigData> = &mut
        ctx.accounts.system_config_account;
29     system_config_data.admin = ctx.accounts.admin.key();
30     system_config_data.count = 0;
31     system_config_data.referral_usage_max_count = 20;
32     system_config_data.protocol_fee_rate = 0.05;
33     system_config_data.protocol_fee_account = ctx.accounts.admin.key();
34     system_config_data.refund_fee_rate = 0.05;
35     system_config_data.referrer_reset_interval_seconds = 86400;
36     system_config_data.update_metadata_fee = 100_000_000;
37     system_config_data.customized_deploy_fee = 150_000_000;
38     system_config_data.init_pool_wsol_amount = 100_000; // 100_000 means
        100% of wsol in vault, 50_000 means 50%.
39     system_config_data.graduate_fee_rate = 5;
40     system_config_data.min_graduate_fee = 5_000_000_000;
41     system_config_data.raydium_cpmm_create_fee = 200_000;
42     system_config_data.transfer_fee_basis_points = 9_000;
43     system_config_data.is_pause = false;
44     Ok(())
45 }
```

/src/instructions/mint_tokens.rs

```
321 // 10- Update graduate epoch, only set once. That's why we use u32::MAX
        to check.
322 config_data.mint_state_data.current_era = get_era_by_supply(config_data,
        config_data.mint_state_data.supply)?;
323 if config_data.mint_state_data.current_era > config_data.target_eras &&
        config_data.mint_state_data.graduate_epoch == u32::MAX {
324     config_data.mint_state_data.graduate_epoch = Clock::get()?.epoch as
        u32;
325     // Send graduate fee from wsol vault to system fee vault
```

```

326     let wsol_vault_balance = get_associated_token_account_balance(&ctx.
accounts.wsol_vault.to_account_info());
327     let graduate_fee = (wsol_vault_balance as u128 * ctx.accounts.
system_config_account.graduate_fee_rate as u128 / 100) as u64;
328     let graduate_fee = if graduate_fee < ctx.accounts.
system_config_account.min_graduate_fee { ctx.accounts.
system_config_account.min_graduate_fee } else { graduate_fee };
329     let init_pool_wsol_amount = (wsol_vault_balance as u128 * ctx.
accounts.system_config_account.init_pool_wsol_amount as u128 /
100_000) as u64;
330     // msg!("graduate_fee: {}", graduate_fee);
331     // msg!("wsol_vault_balance: {}", wsol_vault_balance);
332     // msg!("raydium_cpmm_create_fee: {}", ctx.accounts.
system_config_account.raydium_cpmm_create_fee);
333     // msg!("init_pool_wsol_amount: {}", init_pool_wsol_amount);
334     require!(graduate_fee + ctx.accounts.system_config_account.
raydium_cpmm_create_fee + init_pool_wsol_amount <=
wsol_vault_balance, ErrorCode::GraduateFeeIsNotEnough);

```

Recommendation

It is recommended to limit `system_config_data.init_pool_wsol_amount` to be less than `100,000`.

Alleviation

The project team acknowledged the issue and decided to keep no change. They stated that the need to configure correct parameters via the `update_system` instruction before the system's official launch is expected behavior.

L-08: initialize_system and initialize_launch_rule Susceptible to Front-Running



Low: Business Security

File Location:

/src/instructions/initialize_launch_rule.rs:16,21

/src/instructions/initialize_system.rs:15,20

Description

Within the constraints for the `system_config_account`, the check `constraint = system_config_account.admin == Pubkey::default() || system_config_account.admin == admin.key() @ ErrorCode::SystemAlreadyInitialized` is implemented to ensure the `initialize_system` function is either being called for the first time or is being called by the existing `admin`.

However, if an attacker front-runs the legitimate administrator's transaction and calls `initialize_system` right after the contract is deployed, they can seize control of the contract by setting themselves as the `admin`.

Same vulnerability exists in the `initialize_launch_rule` function.

/src/instructions/initialize_launch_rule.rs

```
8 pub struct InitializeLaunchRule<'info> {
9     #[account(
10         mut,
11         constraint = admin.key() != Pubkey::default() @
12             ErrorCode::OnlySystemAdminAllowed,
13     )]
14     pub admin: Signer<'info>,
15     #[account(
16         init_if_needed,
17         seeds = [LAUNCH_RULE_SEEDS.as_bytes(), SYSTEM_CONFIG_CREATOR.as_ref()],
18         bump,
19         payer = admin,
20         space = 8 + LaunchRuleData::INIT_SPACE,
21         constraint = launch_rule_data.admin == Pubkey::default() ||
22             launch_rule_data.admin == admin.key() @
23             ErrorCode::LaunchRuleAlreadyInitialized,
24     )]
25     pub launch_rule_data: Box<Account<'info, LaunchRuleData>>,
26     pub system_program: Program<'info, System>,
27 }
```

```
7 pub struct InitializeSystem<'info> {
8     #[account(
9         mut,
10        constraint = admin.key() != Pubkey::default() @
            ErrorCode::OnlySystemAdminAllowed,
11        constraint = admin.is_signer @ ErrorCode::OnlySystemAdminAllowed,
12    )]
13    pub admin: Signer<'info>,
14    #[account(
15        init_if_needed,
16        seeds = [SYSTEM_CONFIG_SEEDS.as_bytes(), SYSTEM_CONFIG_CREATOR.as_ref
            ()],
17        bump,
18        payer = admin,
19        space = 8 + SystemConfigData::INIT_SPACE,
20        constraint = system_config_account.admin == Pubkey::default() ||
            system_config_account.admin == admin.key() @
            ErrorCode::SystemAlreadyInitialized,
21    )]
22    pub system_config_account: Box<Account<'info, SystemConfigData>>,
23    pub system_program: Program<'info, System>,
24 }
```

Recommendation

It is recommended to hardcode an `admin` public key within the contract and restrict the `initialize_system` and `initialize_launch_rule` functions to be callable only by this address.

Additionally, the account constraints for `system_config_account` and `launch_rule_data` should be changed from `init_if_needed` to `init`.

Alleviation

Resolved in commit 0ef6247.

L-09: Lack of Upper Bound for Fee Rate

Low: Business Security



File Location:

/src/instructions/initialize_system.rs:32,34,39
/src/instructions/update_system.rs:57,58,63

Description

In the `initialize_system` function, the `protocol_fee_rate`, `refund_fee_rate`, and `graduate_fee_rate` parameters should be constrained by an upper bound to prevent them from being set to excessively high values, which would be detrimental to user interests.

Same vulnerability exists in the `update_system` function.

/src/instructions/initialize_system.rs

```
27 pub fn initialize_system(ctx: Context<InitializeSystem>) -> Result<()> {
28     let system_config_data: &mut Account<'_, SystemConfigData> = &mut
        ctx.accounts.system_config_account;
29     system_config_data.admin = ctx.accounts.admin.key();
30     system_config_data.count = 0;
31     system_config_data.referral_usage_max_count = 20;
32     system_config_data.protocol_fee_rate = 0.05;
33     system_config_data.protocol_fee_account = ctx.accounts.admin.key();
34     system_config_data.refund_fee_rate = 0.05;
35     system_config_data.referrer_reset_interval_seconds = 86400;
36     system_config_data.update_metadata_fee = 100_000_000;
37     system_config_data.customized_deploy_fee = 150_000_000;
38     system_config_data.init_pool_wsol_amount = 100_000; // 100_000 means
        100% of wsol in vault, 50_000 means 50%.
39     system_config_data.graduate_fee_rate = 5;
40     system_config_data.min_graduate_fee = 5_000_000_000;
41     system_config_data.raydium_cpmm_create_fee = 200_000;
42     system_config_data.transfer_fee_basis_points = 9_000;
43     system_config_data.is_pause = false;
44     Ok(())
45 }
```

/src/instructions/update_system.rs

```
25 pub fn update_system(
26     ctx: Context<UpdateSystem>,
27     admin: Pubkey,
28     referral_usage_max_count: u32,
29     protocol_fee_rate: u64,
30     protocol_fee_account: Pubkey,
31     refund_fee_rate: u64,
32     referrer_reset_interval_seconds: u64,
33     update_metadata_fee: u64,
34     customized_deploy_fee: u64,
35     init_pool_wsol_amount: u64,
36     graduate_fee_rate: u64,
37     min_graduate_fee: u64,
```

```

38     raydium_cpmm_create_fee: u64,
39     transfer_fee_basis_points: u16,
40     is_pause: bool,
41 ) -> Result<()> {
42     let system_config_data: &mut Account<'_, SystemConfigData> = &mut
        ctx.accounts.system_config_account;
43     require!(protocol_fee_account != Pubkey::default(),
        ErrorCode::InvalidFeeAccount);
44     require!(referrer_reset_interval_seconds > 0,
        ErrorCode::InvalidResetInterval);
45     // require!(protocol_fee_rate <= 50_000,
        ErrorCode::InvalidProtocolFeeRate);
46     // require!(refund_fee_rate <= 50_000,
        ErrorCode::InvalidRefundFeeRate);
47
48     // Additional validation for admin transfer
49     if admin != ctx.accounts.admin.key() {
50         // Log admin change for security monitoring
51         msg!("Admin changed from {} to {}", ctx.accounts.admin.key(),
            admin);
52     }
53
54     system_config_data.admin = admin;
55     system_config_data.referral_usage_max_count =
        referral_usage_max_count;
56     system_config_data.protocol_fee_account = protocol_fee_account;
57     system_config_data.protocol_fee_rate = protocol_fee_rate.safe_as_f64
        ()? / 100_000.0;
58     system_config_data.refund_fee_rate = refund_fee_rate.safe_as_f64
        ()? / 100_000.0;
59     system_config_data.referrer_reset_interval_seconds =
        referrer_reset_interval_seconds;
60     system_config_data.update_metadata_fee = update_metadata_fee;
61     system_config_data.customized_deploy_fee = customized_deploy_fee;
62     system_config_data.init_pool_wsol_amount = init_pool_wsol_amount;
63     system_config_data.graduate_fee_rate = graduate_fee_rate;
64     system_config_data.min_graduate_fee = min_graduate_fee;
65     system_config_data.raydium_cpmm_create_fee = raydium_cpmm_create_fee;
66     system_config_data.transfer_fee_basis_points =
        transfer_fee_basis_points;
67     system_config_data.is_pause = is_pause;
68     Ok(())
69 }

```

Recommendation

It is recommended to set upper bounds for `protocol_fee_rate`, `refund_fee_rate`, and `graduate_fee_rate` to increase project transparency.

Alleviation

Resolved in commit 0ef6247.

L-10: Insufficient Validation

Low: Business Security



File Location:

/src/instructions/proxy_burn_lp_tokens.rs:51,55
/src/instructions/proxy_deposit_pool.rs:34,55
/src/instructions/proxy_swap_base_in_pool.rs /src/instructions/proxy_swap_base_out_pool.rs:75,82,85,88
/src/instructions/proxy_withdraw_pool.rs:34,55

Description

There are multiple instances of insufficient validation in the project.

1. In the `ProxyDeposit` and `ProxyWithdraw` structs, there is no validation to check if either `pool_state.token_0_mint` or `pool_state.token_1_mint` is the same as the `mint` account. As a result, a caller could pass in a mismatched `pool_state`.
2. In the `ProxyBurnLPTokens` struct, the association between the `lp_mint` and `owner_lp_token` accounts is not validated.
3. In the `ProxySwapBaseInput` and `ProxySwapBaseOutput` structs, there is no validation to ensure that `input_vault` and `output_vault` are not equal. Additionally, there is no validation to ensure that `input_token_program` and `output_token_program` are not equal.

/src/instructions/proxy_burn_lp_tokens.rs

```
46  /// Owner lp token account - config account
47  #[account(
48    mut,
49    token::authority = config_account
50  )]
51  pub owner_lp_token: Box<InterfaceAccount<'info, TokenAccount>>,
52
53  /// pool lp mint, init by cp-swap
54  #[account(mut)]
55  pub lp_mint: Box<InterfaceAccount<'info, Mint>>,
```

/src/instructions/proxy_deposit_pool.rs

```
29  /// Token mint
30  #[account(
31    seeds = [MINT_SEEDS.as_bytes(), token_name.as_bytes(), token_symbol.
32              to_lowercase().as_bytes()],
33    bump,
34  )]
35  pub mint: Box<InterfaceAccount<'info, Mint>>,
36
37  /// Config account
38  #[account(
39    mut,
40    seeds = [CONFIG_SEEDS.as_bytes(), mint.key().as_ref()],
41    bump,
42    constraint = !config_account.is_processing @
43    ErrorCode::DepositInProgress.
```

```

42  })
43  pub config_account: Box<Account<'info, TokenConfigData>>,
44
45  /// CHECK: pool vault and lp mint authority
46  #[account(
47      seeds = [AUTH_SEED.as_bytes()],
48      bump,
49      seeds::program = RAYDIUM_CP_SWAP_PROGRAM_ID,
50  )]
51  pub authority: UncheckedAccount<'info>,
52
53  /// pool state
54  #[account(mut)]
55  pub pool_state: AccountLoader<'info, PoolState>,

```

/src/instructions/proxy_swap_base_in_pool.rs

/src/instructions/proxy_swap_base_out_pool.rs

```

70  /// The vault token account for input token
71  #[account(
72      mut,
73      constraint = input_vault.key() == pool_state.load()?.
        token_0_vault || input_vault.key() == pool_state.load()?.
        token_1_vault
74  )]
75  pub input_vault: Box<InterfaceAccount<'info, TokenAccount>>,
76
77  /// The vault token account for output token
78  #[account(
79      mut,
80      constraint = output_vault.key() == pool_state.load()?.
        token_0_vault || output_vault.key() == pool_state.load()?.
        token_1_vault
81  )]
82  pub output_vault: Box<InterfaceAccount<'info, TokenAccount>>,
83
84  /// SPL program for input token transfers
85  pub input_token_program: Interface<'info, TokenInterface>,
86
87  /// SPL program for output token transfers
88  pub output_token_program: Interface<'info, TokenInterface>,

```

/src/instructions/proxy_withdraw_pool.rs

```

29  /// Token mint
30  #[account(
31      seeds = [MINT_SEEDS.as_bytes(), token_name.as_bytes(), token_symbol.
        to_lowercase().as_bytes()],
32      bump,
33  )]
34  pub mint: Box<InterfaceAccount<'info, Mint>>,
35
36  /// Config account
37  #[account(
38      mut,

```



```

39 seeds = [CONFIG_SEEDS.as_bytes(), mint.key().as_ref()],
40 bump,
41 constraint = !config_account.is_processing @
  ErrorCode::WithdrawInProgress,
42 )]
43 pub config_account: Box<Account<'info, TokenConfigData>>,
44
45 /// CHECK: pool vault and lp mint authority
46 #[account(
47     seeds = [AUTH_SEED.as_bytes()],
48     bump,
49     seeds::program = RAYDIUM_CP_SWAP_PROGRAM_ID,
50 )]
51 pub authority: UncheckedAccount<'info>,
52
53 /// pool state
54 #[account(mut)]
55 pub pool_state: AccountLoader<'info, PoolState>,

```

Recommendation

It is recommended to implement the aforementioned validation checks.

Alleviation

The project team acknowledged the issue and decided to keep no change. They stated that further account validation will be performed within the Raydium CPI program.

I-11: Typographical Error



Informational: Optimization Suggestion

File Location: /src/utls.rs:93

Description

The word "vault" is misspelled as "valut" in the function name `get_mint_size_of_valut`.

/src/utls.rs

```
93 pub fn get_mint_size_of_valut(  
94     config_data: &TokenConfigData,  
95     mint_size: u64,  
96 ) -> Result<u64> {  
97     let mint_size_for_vault = (mint_size as f64 * config_data.  
        liquidity_tokens_ratio / (1.0 - config_data.liquidity_tokens_ratio)).  
        safe_as_u64()?;  
98     Ok(mint_size_for_vault)  
99 }
```

Recommendation

It is recommended to rename the function `get_mint_size_of_valut` to `get_mint_size_of_vault`.

Alleviation

Resolved in commit 0ef6247.

I-12: Mismatch Between Code Logic and Error Message



Informational: Optimization Suggestion

File Location: /src/instructions/refund.rs:123

Description

The conditional check uses a "greater than or equal to" ($\geq$) comparison, while the error message implies a "not equal to" condition, creating an inconsistency.

/src/instructions/refund.rs

```
119 let burn_amount_from_user = refund_data.total_tokens;
120 let burn_amount_from_vault = refund_data.vault_tokens;
121 require!(mint.supply >= burn_amount_from_user + burn_amount_from_vault,
    ErrorCode::NotEnoughTokensToRefund);
122 require!(token_ata.amount == burn_amount_from_user,
    ErrorCode::UserBalanceNotEqualToRefund);
123 require!(ctx.accounts.token_vault.amount >= burn_amount_from_vault,
    ErrorCode::VaultBalanceNotEqualToRefund);
```

Recommendation

It is recommended to align the code logic with the error message.

Alleviation

Resolved in commit 0ef6247.

I-13: Unnecessary Check in mint_tokens Function



Informational: Optimization Suggestion

File Location: /src/instructions/mint_tokens.rs:69,74,76,143,144,149,150,163,164,165,166,167

Description

Some of the checks within the `mint_tokens` function are redundant, as they duplicate constraints already defined in the `MintTokens<'info>` struct.

/src/instructions/mint_tokens.rs

```
65  #[account(  
66      mut,  
67      constraint = mint_token_vault.key() == config_account.  
        mint_token_vault @ ErrorCode::InvalidMintTokenVault,  
68      constraint = mint_token_vault.mint == mint.key() @  
        ErrorCode::InvalidTokenMint,  
69      constraint = mint_token_vault.owner == mint.key() @  
        ErrorCode::InvalidMintTokenVaultOwner,  
70  )]  
71  pub mint_token_vault: Box<Account<'info, TokenAccount>>,  
72  #[account(  
73      mut,  
74      constraint = token_vault.key() == config_account.token_vault @  
        ErrorCode::InvalidTokenVault,  
75      constraint = token_vault.mint == mint.key() @  
        ErrorCode::InvalidTokenMint,  
76      constraint = token_vault.owner == config_account.key() @  
        ErrorCode::InvalidTokenVaultOwner,  
77  )]  
78  pub token_vault: Box<Account<'info, TokenAccount>>,
```

/src/instructions/mint_tokens.rs

```
141  // token 0 mint  
142  #[account(  
143      constraint = (token_0_mint.key() == mint.key() && token_1_mint.key() ==  
        WSOL)  
144      || (token_1_mint.key() == mint.key() && token_0_mint.key()  
        () == WSOL) @ ErrorCode::WrongPoolToken,  
145  )]  
146  pub token_0_mint: Box<Account<'info, Mint>>,  
147  // token 1 mint  
148  #[account(  
149      constraint = (token_0_mint.key() == mint.key() && token_1_mint.key() ==  
        WSOL)  
150      || (token_1_mint.key() == mint.key() && token_0_mint.key()  
        () == WSOL) @ ErrorCode::WrongPoolToken,  
151  )]  
152  pub token_1_mint: Box<Account<'info, Mint>>,
```

```
161 pub fn mint_tokens(mut ctx: Context<MintTokens>, token_name: String,  
162 token_symbol: String, code_hash: Vec<u8>) -> Result<()> {  
163     require!(spl_token::ID == ctx.accounts.token_program.key(),  
164         ErrorCode::WrongTokenProgram);  
165     require!(ctx.accounts.token_vault.key() == ctx.accounts.  
166         config_account.token_vault.key(), ErrorCode::InvalidTokenVault);  
167     require!(ctx.accounts.token_vault.owner.key() == ctx.accounts.  
168         config_account.key(), ErrorCode::InvalidTokenVaultOwner);  
169     require!(ctx.accounts.mint_token_vault.owner.key() == ctx.accounts.  
170         mint.key(), ErrorCode::InvalidMintTokenVaultOwner);  
171     require!(( ctx.accounts.token_0_mint.key() == ctx.accounts.mint.key  
172         () && ctx.accounts.token_1_mint.key() == WSOL )  
173         || ( ctx.accounts.token_1_mint.key() == ctx.accounts.mint.key() &&  
174             ctx.accounts.token_0_mint.key() == WSOL ),  
175         ErrorCode::WrongPoolToken);  
176     require!(ctx.accounts.token_0_mint.key() < ctx.accounts.token_1_mint.  
177         key(), ErrorCode::WrongTokenMintOrder);
```

Recommendation

It is recommended to remove these superfluous checks from `mint_tokens` function.

Alleviation

Resolved in commit 0ef6247.

I-14: Adherence to the Checks-Effects-Interactions Pattern



Informational: Optimization Suggestion

File Location: /src/instructions/mint_tokens.rs:214,215

Description

It is recommended to perform conditional checks before making state modifications to prevent unnecessary code execution. For example, the `_supply <= config_data.max_supply` check should be performed before modifying

```
config_data.mint_state_data.quantity_minted_epoch .
```

/src/instructions/mint_tokens.rs

```
211 // 3- Added supply should include the liquidity tokens
212 let mint_size_for_vault = get_mint_size_of_valut(config_data,
    config_data.mint_state_data.mint_size_epoch?);
213 let _supply = config_data.mint_state_data.supply + config_data.
    mint_state_data.mint_size_epoch + mint_size_for_vault; // including
    vault tokens
214 config_data.mint_state_data.quantity_minted_epoch += config_data.
    mint_state_data.mint_size_epoch + mint_size_for_vault; // including
    vault tokens
215 require!(_supply <= config_data.max_supply, ErrorCode::ExceedMaxSupply);
216 config_data.mint_state_data.supply = _supply;
```

Recommendation

It is recommended to follow the Checks-Effects-Interactions pattern.

Alleviation

Resolved in commit 0ef6247.

I-15: Redundant Check

Informational: Optimization Suggestion

File Location:

 /src/instructions/burn_mint_token_vault.rs:24
/src/instructions/close_token.rs:20,21
/src/instructions/delegate_value_manager.rs:14
/src/instructions/initialize_launch_rule.rs:11,12
/src/instructions/initialize_system.rs:10,11
/src/instructions/initialize_token.rs:38,39
/src/instructions/mint_tokens.rs:48,49
/src/instructions/refund.rs:60,61
/src/instructions/revoke_update_metadata_authority.rs:36,37
/src/instructions/set_referrer_code.rs:16,17
/src/instructions/update_system.rs:10,11
/src/instructions/update_token_metadata.rs:28,29

Description

Anchor automatically verifies if an account of type `Signer<'info>` is a signer, which makes the explicit `constraint = xxx.is_signer` and `xxx.key() != Pubkey::default()` checks redundant.

/src/instructions/burn_mint_token_vault.rs

```
21  #[account(  
22      mut,  
23      constraint = payer.key() == config_account.admin @  
          ErrorCode::OnlyAdminAllowed,  
24      constraint = payer.is_signer @ ErrorCode::PayerMustSign  
25  )]  
26  pub payer: Signer<'info>,
```

/src/instructions/close_token.rs

```
18  #[account(  
19      mut,  
20      constraint = payer.key() != Pubkey::default() @  
          ErrorCode::InvalidPayerAccount,  
21      constraint = payer.is_signer @ ErrorCode::PayerMustSign  
22  )]  
23  pub payer: Signer<'info>,
```

/src/instructions/delegate_value_manager.rs

```
11  #[account(  
12      mut,  
13      constraint = admin.key() == config_account.admin.key() @  
          ErrorCode::OnlyValueManagerAllowed,  
14      constraint = admin.is_signer @ ErrorCode::PayerMustSign,  
15  )]  
16  pub admin: Signer<'info>,
```

/src/instructions/initialize_launch_rule.rs

```
9  #[account(  
10      mut,  
11      constraint = admin.key() != Pubkey::default() @  
12      ErrorCode::OnlySystemAdminAllowed,  
13      constraint = admin.is_signer @ ErrorCode::OnlySystemAdminAllowed,  
14  )]  
14  pub admin: Signer<'info>,
```

/src/instructions/initialize_system.rs

```
8  #[account(  
9      mut,  
10     constraint = admin.key() != Pubkey::default() @  
11     ErrorCode::OnlySystemAdminAllowed,  
12     constraint = admin.is_signer @ ErrorCode::OnlySystemAdminAllowed,  
13 )]  
13 pub admin: Signer<'info>,
```

/src/instructions/initialize_token.rs

```
36  #[account(  
37      mut,  
38     constraint = payer.key() != Pubkey::default() @  
39     ErrorCode::InvalidPayerAccount,  
40     constraint = payer.is_signer @ ErrorCode::PayerMustSign,  
41 )]  
41 pub payer: Signer<'info>,
```

/src/instructions/mint_tokens.rs

```
46  #[account(  
47      mut,  
48     constraint = user.key() != Pubkey::default() @  
49     ErrorCode::InvalidPayerAccount,  
50     constraint = user.is_signer @ ErrorCode::PayerMustSign  
51 )]  
51 pub user: Signer<'info>,
```

/src/instructions/refund.rs

```
58  #[account(  
59      mut,  
60     constraint = payer.key() != Pubkey::default() @  
61     ErrorCode::InvalidPayerAccount,  
62     constraint = payer.is_signer @ ErrorCode::PayerMustSign  
63 )]  
63 pub payer: Signer<'info>,
```


/src/instructions/revoke_update_metadata_authority.rs

```
34  #[account(  
35      mut,  
36      constraint = payer.key() != Pubkey::default() @  
          ErrorCode::InvalidPayerAccount,  
37      constraint = payer.is_signer @ ErrorCode::PayerMustSign,  
38  )]  
39  pub payer: Signer<'info>,
```

/src/instructions/set_referrer_code.rs

```
14  #[account(  
15      mut,  
16      constraint = payer.key() != Pubkey::default() @  
          ErrorCode::InvalidPayerAccount,  
17      constraint = payer.is_signer @ ErrorCode::PayerMustSign  
18  )]  
19  pub payer: Signer<'info>,
```

/src/instructions/update_system.rs

```
8  #[account(  
9      mut,  
10     constraint = admin.key() != Pubkey::default() @  
        ErrorCode::OnlySystemAdminAllowed,  
11     constraint = admin.is_signer @ ErrorCode::OnlySystemAdminAllowed  
12 )]  
13 pub admin: Signer<'info>,
```

/src/instructions/update_token_metadata.rs

```
26  #[account(  
27      mut,  
28      constraint = payer.key() != Pubkey::default() @  
          ErrorCode::InvalidPayerAccount,  
29      constraint = payer.is_signer @ ErrorCode::PayerMustSign,  
30      constraint = payer.to_account_info().lamports() >  
          system_config_account.update_metadata_fee @  
          ErrorCode::PayerAmountTooLow  
31  )]  
32  pub payer: Signer<'info>,
```

Recommendation

It is recommended to remove the `constraint = xxx.is_signer` and `xxx.key() != Pubkey::default()` checks for accounts of type `Signer<'info>`.

Alleviation

Resolved in commit 0ef6247.

I-16: Lack of Zero-Address Check



Informational: Optimization Suggestion

File Location: /src/instructions/update_system.rs:27

Description

The `update_system` function does not validate if the `admin` parameter is equal to `Pubkey::default()`. Incorrectly setting the `admin` to this default value would render privileged functions unusable.

/src/instructions/update_system.rs

```
25 pub fn update_system(  
26     ctx: Context<UpdateSystem>,  
27     admin: Pubkey,  
28     referral_usage_max_count: u32,  
29     protocol_fee_rate: u64,  
30     protocol_fee_account: Pubkey,  
31     refund_fee_rate: u64,  
32     referrer_reset_interval_seconds: u64,  
33     update_metadata_fee: u64,  
34     customized_deploy_fee: u64,  
35     init_pool_wsol_amount: u64,  
36     graduate_fee_rate: u64,  
37     min_graduate_fee: u64,  
38     raydium_cpmm_create_fee: u64,  
39     transfer_fee_basis_points: u16,  
40     is_pause: bool,  
41 ) -> Result<()> {  
42     let system_config_data: &mut Account<'_, SystemConfigData> = &mut  
43         ctx.accounts.system_config_account;  
44     require!(protocol_fee_account != Pubkey::default(),  
45         ErrorCode::InvalidFeeAccount);  
46     require!(referrer_reset_interval_seconds > 0,  
47         ErrorCode::InvalidResetInterval);  
48     // require!(protocol_fee_rate <= 50_000,  
49         ErrorCode::InvalidProtocolFeeRate);  
50     // require!(refund_fee_rate <= 50_000,  
51         ErrorCode::InvalidRefundFeeRate);  
52     // Additional validation for admin transfer  
53     if admin != ctx.accounts.admin.key() {  
54         // Log admin change for security monitoring  
55         msg!("Admin changed from {} to {}", ctx.accounts.admin.key(),  
56             admin);  
57     }  
58 }
```

Recommendation

It is recommended to add the following check: `require!(admin != Pubkey::default(), ErrorCode::InvalidAdmin);`.

Alleviation

Resolved in commit 0ef6247.

4. Disclaimer

No description, statement, recommendation or conclusion in this report shall be construed as endorsement, affirmation or confirmation of the project. The security assessment is limited to the scope of work as stipulated in the Statement of Work.

This report is prepared in response to source code, and based on the attacks and vulnerabilities in the source code that already existed or occurred before the date of this report, excluding any new attacks or vulnerabilities that exist or occur after the date of this report. The security assessment are solely based on the documents and materials provided by the customer, and the customer represents and warrants documents and materials are true, accurate and complete.

CONSULTANT DOES NOT MAKE AND HEREBY DISCLAIMS ANY REPRESENTATIONS OR WARRANTIES OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY OR OTHERWISE, REGARDING THE SERVICES, DELIVERABLES, OR ANY OTHER MATTER PERTAINING TO THIS REPORT.

CONSULTANT SHALL NOT BE RESPONSIBLE FOR AND HEREBY DISCLAIMS MERCHANTABILITY, FITNESS FOR PURPOSE, TITLE, NON-INFRINGEMENT OR NON-APPROPRIATION OF INTELLECTUAL PROPERTY RIGHTS OF A THIRD PARTY, SATISFACTORY QUALITY, ACCURACY, QUALITY, COMPLETENESS, TIMELINESS, RESPONSIVENESS, OR PRODUCTIVITY OF THE SERVICES OR DELIVERABLES.

CONSULTANT EXCLUDES ANY WARRANTY THAT THE SERVICES AND DELIVERABLES WILL BE UNINTERRUPTED, ERROR FREE, FREE OF SECURITY DEFECTS OR HARMFUL COMPONENTS, REVEAL ALL SECURITY VULNERABILITIES, OR THAT ANY DATA WILL NOT BE LOST OR CORRUPTED.

CONSULTANT SHALL NOT BE RESPONSIBLE FOR (A) ANY REPRESENTATIONS MADE BY ANY PERSON REGARDING THE SUFFICIENCY OR SUITABILITY OF SERVICES AND DELIVERABLES IN ANY ACTUAL APPLICATION, OR (B) WHETHER ANY SUCH USE WOULD VIOLATE OR INFRINGE THE APPLICABLE LAWS, OR (C) REVIEWING THE CUSTOMER MATERIALS FOR ACCURACY.